Module 3: Token Standards (SPL Tokens)

Solana Program Library Tokens

SPL vs ERC Standards

Solana Token Standards

- **SPL (Solana Program Library)**: Native token standard
- **SPL-Token**: Fungible tokens (like ERC-20)
- **SPL-Token-2022**: Enhanced token standard with extensions

EVM Comparison

- **ERC-20**: Basic fungible tokens
- **ERC-721**: Non-fungible tokens
- **ERC-1155**: Multi-token standard

Token Account Model

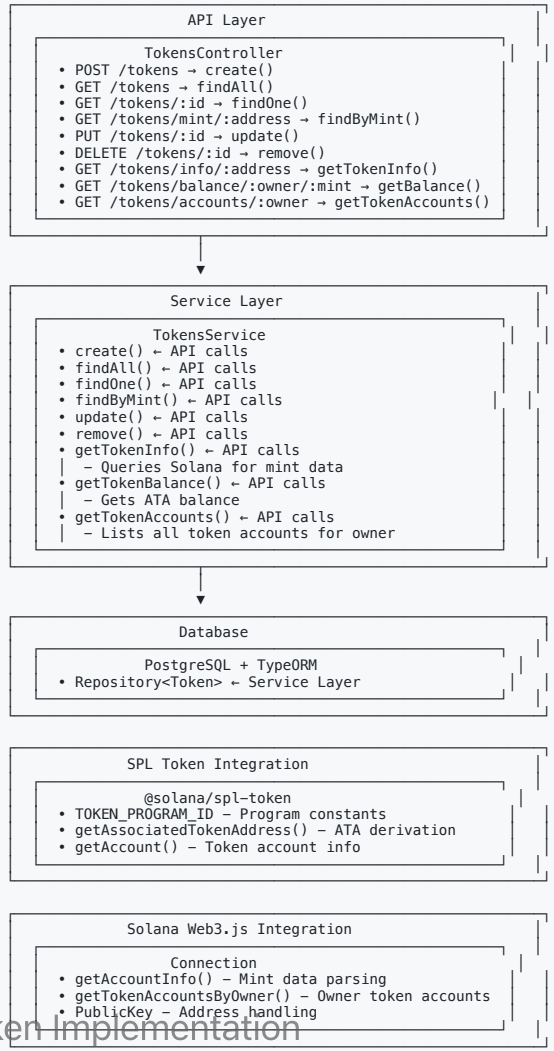
Three Account Types

1. **Mint Account**: Token definition (supply, decimals, authorities)
2. **Token Account**: User's token balance
3. **Associated Token Account (ATA)**: Deterministic token account address

Key Differences from EVM

- **Separate Accounts**: Each token holding requires a token account
- **ATAs**: Automatic account derivation per wallet-token pair
- **No Direct Balance**: Balance stored in separate token accounts

Architecture Overview



Token Operations

Creating a Token

```
// Create mint account
const mint = await createMint(
  connection,
  payer,
  mintAuthority,
  freezeAuthority,
  decimals
);

// Create token account
const tokenAccount = await getOrCreateAssociatedTokenAccount(
  connection,
  payer,
  mint,
  owner
);
```

Token Transfer

```
// Transfer tokens between accounts
await transfer(
  connection,
  payer,
  sourceTokenAccount,
  destinationTokenAccount,
  owner,
  amount
);
```

Minting Tokens

```
// Mint new tokens to account
await mintTo(
  connection,
  payer,
  mint,
  destinationTokenAccount,
  mintAuthority,
  amount
);
```

SPL Token Extensions

Token-2022 Features

- **Transfer Fees**: Charge fees on transfers
- **Confidential Transfers**: Private token amounts
- **Non-transferable Tokens**: Soul-bound tokens
- **Interest-bearing Tokens**: Automatic yield
- **Permanent Delegate**: Always-authorized delegate

Metadata Extension

- **Token Metadata**: Name, symbol, description, image
- **Creator Verification**: Verify content creators
- **Royalties**: Automatic royalty payments

NFT Implementation

SPL Token NFTs

- **Supply = 1**: Single token per mint
- **Decimals = 0**: No fractional NFTs
- **Unique Metadata**: Each NFT has unique attributes

Metaplex Standard

- **Metadata Account**: Stores NFT metadata
- **Master Edition**: Controls supply and royalties
- **Creator Verification**: Verify NFT creators

Common Patterns

Token Balance Checking

```
// Get token balance
const tokenAccounts = await getTokenAccountsByOwner(
  connection,
  ownerPublicKey,
  { mint: mintPublicKey }
);

const balance = tokenAccounts.value[0]?.account.data.parsed.info.tokenAmount.uiAmount;
```

Batch Token Operations

```
// Create multiple token transfers
const instructions = [];
for (const transfer of transfers) {
  const ix = createTransferInstruction(
    transfer.source,
    transfer.destination,
    transfer.owner,
    transfer.amount
  );
  instructions.push(ix);
}

// Execute all in one transaction
const transaction = new Transaction().add(...instructions);
```

Performance Considerations

Optimization Strategies

- **ATA Caching**: Cache associated token addresses
- **Batch Operations**: Combine multiple transfers
- **Lazy Loading**: Load token data on demand
- **Index Optimization**: Efficient database queries

Scalability

- **Parallel Processing**: Handle multiple token operations
- **Connection Pooling**: Efficient RPC connections
- **Rate Limiting**: Prevent API abuse

Security Best Practices

Token Security

- **Authority Validation**: Verify mint/freeze authorities
- **Amount Validation**: Prevent overflow/underflow
- **Account Ownership**: Verify token account ownership
- **Freeze Protection**: Handle frozen accounts

Smart Contract Risks

- **Reentrancy**: Prevent reentrant calls
- **Access Control**: Proper permission checks
- **Input Validation**: Sanitize all inputs
- **Audit Requirements**: Security audits for custom tokens

Next Steps

Module 3 Implementation Tasks

- ✅ Basic token CRUD operations
- ✅ Token balance queries
- ✅ Token creation/minting
- ✅ Token transfer operations
- ✅ NFT support
- ✅ Token metadata management

Related Modules

- **Module 2**: Transactions & Instructions
- **Module 4**: Account Abstraction
- **Module 10**: CPIs (Cross-Program Invocations)

Thank You!

Questions?

Ready to explore Account Abstraction?

Next: Module 4 - Account Abstraction ➡️